

Complejidad Temporal

Pablo Castro
UNRC - Computabilidad y Complejidad

Tiempo de Ejecución

En computabilidad nos preocupamos por el siguiente problema:

Dado in problema P , es computable?

En complejidad ya trabajamos con problemas computables, la pregunta es:

Hay algoritmos eficientes para resolver P ?

Una cuestión es cuál modelo de computación usamos para evaluar si un algoritmo es eficiente o no?

Tiempo de Ejecución como un Función

Dado un algoritmo M su tiempo de ejecución viene dado por una función:

$$T_M : \mathbb{N} \rightarrow \mathbb{R}^{\geq 0}$$

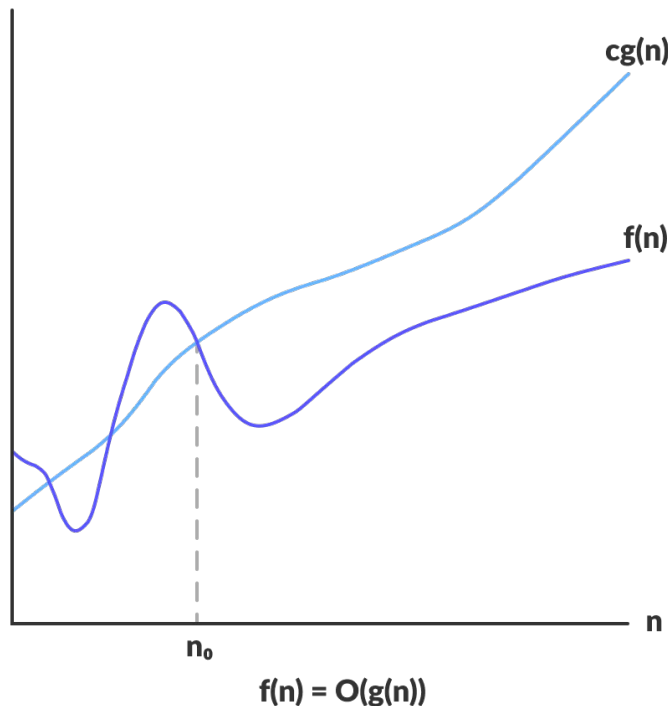
Que dice, para una entrada de tamaño n , cuantos pasos realiza M en el **peor caso**, o **caso promedio**.

- En general asumimos que T es monotonamente creciente,
- M es una MT que termina en todas sus entradas,
- En general hacemos un análisis en el **peor caso**.
- Nos interesa analizar como T crece cuando las entradas se hacen más grandes.

Notación O

La notación O (big oh) nos permite analizar el tiempo de ejecución de los algoritmos.

$$f(n) \in O(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \leq c * g(n)$$



Intuitivamente: *para entradas grandes g crece más rápido que f modulo constantes multiplicativas.*

Notación o (little oh)

La o (little oh) es una versión estricta de la O :

$$f(n) \in o(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) < c * g(n)$$

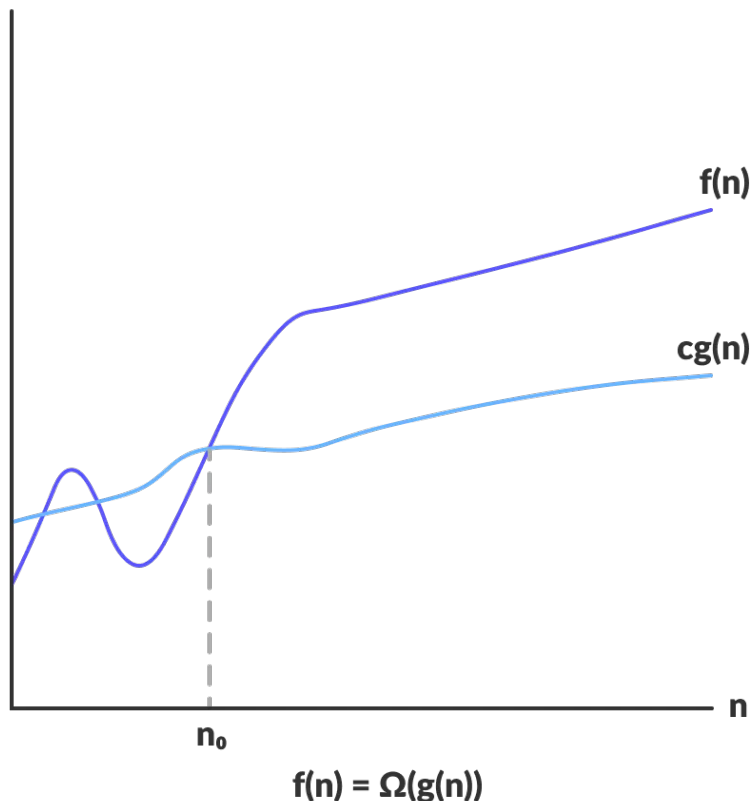
La o dice que $g(n)$ crece estrictamente más rápido que f . Es decir, no tenemos $f(n) \in o(f(n))$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \text{ si } f(n) \in o(g(n))$$

Notación Ω

La Ω (omega) es sirve para hablar de cotas por abajo.

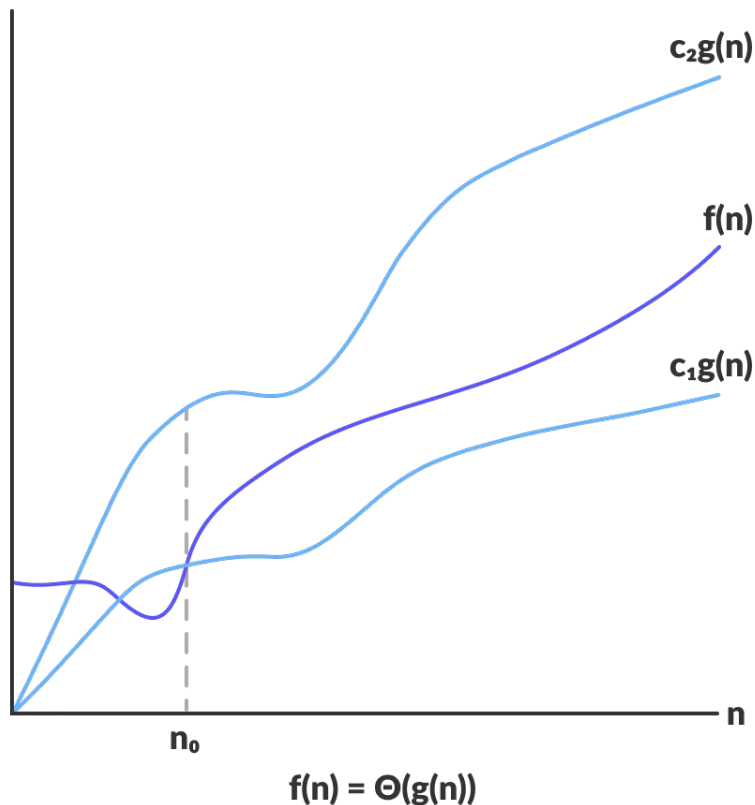
$$f(n) \in \Omega(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \geq c * g(n)$$



Intuitivamente: *para entradas grandes f crece más rápido que g modulo constantes multiplicativas.*

Notación Θ

La Θ (omega) sirve para ver que dos funciones crecen de la misma forma:



Intuitivamente: *para entradas grandes f y g crecen igual modulo constantes multiplicativas.*

Análisis de Tiempo de Ejecución

Supongamos el siguiente lenguaje:

$$A = \{0^k 1^k \mid k \geq 0\}$$

Podemos hacer la siguiente MT para decidirlo.

Dado el input w :

1. Se recorre la cinta, si se encuentra “10” se rechaza,
2. Se vuelve al principio, se tacha el primer “0” y se busca el primer “1” y se tacha,
3. Se repite el paso 2 hasta que:
 - I. No queden 0s y 1s, en tal caso se acepta,
 - II. Queden solo 0s o solo 1s, en tal caso se rechaza

Análisis de Tiempo de Ejecución

Su tiempo ejecución depende de la entrada:

- ➡ Para la entrada $01111\dots1110$ con una pasada se rechaza, es decir, se realizan $O(n)$ pasos.
- ➡ Para entradas del tipo $000\dots000111\dots111$ ($n/2$ 1's y 0s). Hacemos aproximadamente $2 * n + n/2 * 2n$ pasos.

Es decir, en el peor caso es $O(n^2)$ y en el mejor caso es $O(n)$.

En lo siguiente nos enfocaremos en el análisis en el peor caso.

Análisis detallado de Tiempo de Ejecución

Para una entrada de longitud n , analicemos cuántos pasos realiza la máquina:

Dado el input w :

$O(n)$ pasos
($2 * n$)

n pasos

1. Se recorre la cinta, si se encuentra "10" se rechaza,

2. Se vuelve al principio, se tacha el primer "0" y se busca el primer "1" y se tacha,

3. Se repite el paso 2 hasta que:

I. No queden 0s y 1s, en tal caso se acepta,

II. Queden solo 0s o solo 1s, en tal caso se rechaza

$n/2$ veces

En total tenemos $O(n) + O(n/2 * n) \in O(n^2)$ pasos, es decir, es $O(n^2)$

Análisis de Tiempo de Ejecución

La solución es $O(n^2)$, consideremos la siguiente solución:

Dada la entrada w se procede de la siguiente forma:

1. Se recorre la cinta, si se encuentra "10" se rechaza,
2. Repetir mientras haya 0's o 1's
 - a. Se cuenta la cantidad de 0's y 1's, si es impar se rechaza.
 - b. Cada dos 0's tachar de forma alternante 0's, y lo mismo para los 1's,
3. Si no quedan 0's y 1's se acepta, sino se rechaza

Ejemplo de Corrida

Supongamos que tenemos:

000...111... (25 0s y 25 1s)

Entonces:

- ➡ El paso (1) se verifica que está en la forma correcta
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 12 0's y 12 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 6 0's y 6 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 3 0's y 3 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 2 0's y 2 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 1 0's y 1 1's, termina.

Análisis del Algoritmo

Analicemos este algoritmo:

Dada la entrada w se procede de la siguiente forma:

1. Se recorre la cinta, si se encuentra "10" se rechaza,

2. Repetir mientras haya 0's o 1's

I. Se cuenta la cantidad de 0's y 1's, si es impar se rechaza.

II. Cada dos 0's tachar uno de ellos, y lo mismo para los 1's,

3. Si no quedan 0's y 1's se acepta, sino se rechaza

$O(n)$ pasos

$O(\log n)$
pasos

$O(n)$ pasos

En total realizamos $O(n * \log n)$ pasos. Mejor que la otra solución.

Clases de Complejidad

Permite clasificar los problemas en base a cotas superiores de algoritmos que los resuelven:

$\text{Time}(t(n)) = \{L \mid L \text{ se puede decidir con una MT en } O(t(n)) \text{ pasos}\}$

Por ejemplo:

- $\text{Time}(O(n^2))$, problemas para los cuales existen un algoritmo que en el peor caso realizan $c * n^2$ pasos, para alguna constante c (sorting, por ejemplo)

Qué se estudia en complejidad:

- Dado un problema, pertenece a $\text{Time}(O(t(n)))$?
- Habrá problemas que pertenecen a $\text{Time}(O(t(n)))$ pero no a $\text{Time}(O(t'(n)))$

La Clase P

Los algoritmos que corren en $O(n^k)$ para una entrada de longitud n se dicen **tratables**.

	n=10	n=20	n=30	n=40	n=50
n	0.00001 segs	0.00002 segs	0.00003 segs	0.00004 segs	0.00005 segs
n^2	0.0002 segs	0.0004 segs	0.0009 segs	0.0016 segs	0.0025 segs
n^3	0.1 segs	3.2 segs	24.3 segs	1.7 min	15.2 min
2^n	0.001 segs	1.0 segs	17.9 min	12.7 días	35.6 años

Definición de la clase de complejidad P

Podemos definir la clase de los algoritmos cuyo tiempo ejecución es polinomial.

$$P = \bigcup_{k>0} \text{Time}(O(n^k))$$

Es decir, aquellos problemas para los cuales existen algoritmos en $O(n^k)$, para algún k , que los resuelven.

- ✓ La clase P es invariante con respecto a los modelos de computación deterministas.
- ✓ Se consideran eficientes, si tenemos una solución polinomial, en general, el avance en el hardware va a tener un impacto.

Modelos de Computación

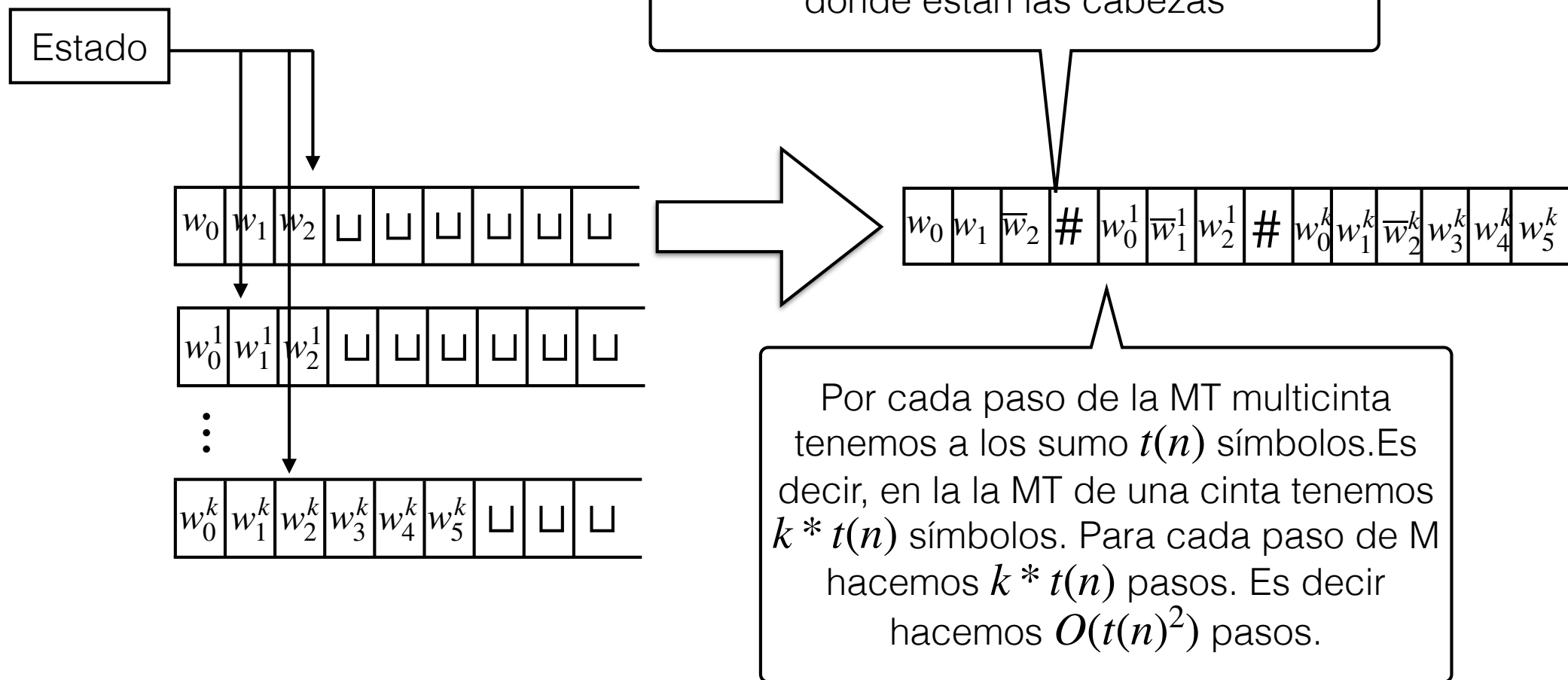
Como hemos visto existen varios modelos de computación:

- Máquinas de Turing deterministas con una sola cinta,
 - Máquinas de Turing deterministas con varias cintas,
 - Máquinas de Turing deterministas con cinta infinita en ambos lados
 - Máquinas de Turing no-deterministas.
- 
- Se pueden simular en tiempo polinomial

La clase Time solo toma como referencia las máquinas deterministas

MT con muchas cintas vs MT con una cinta

Cualquier MT con muchas cintas que hace $O(t(n))$ pasos puede ser traducida a una MT con una cinta que la simula en $O(t(n)^2)$ pasos:



Tesis de Church-Turing Extendida

Recordemos la tesis de Church-Turing.

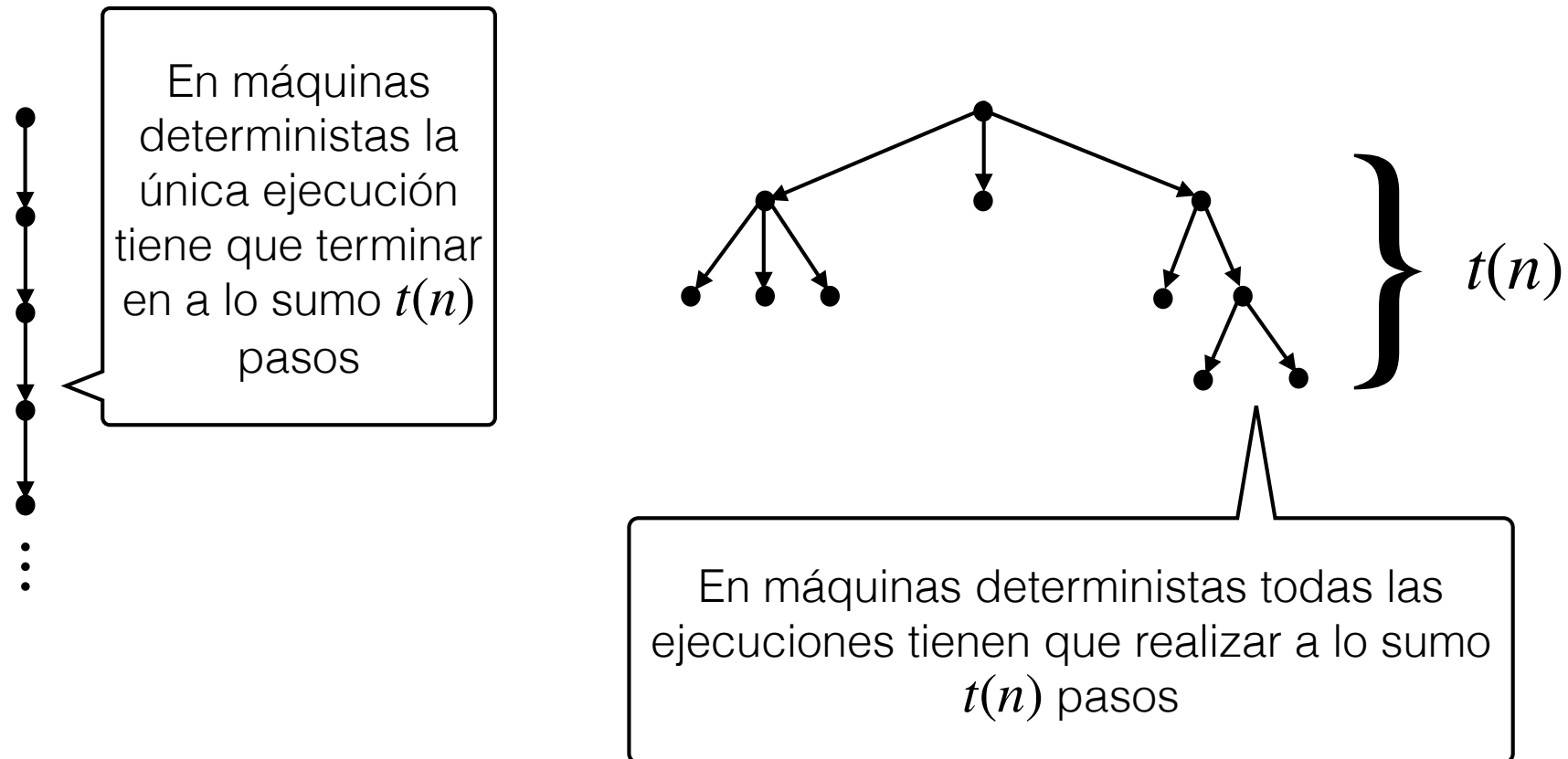
Si un problema es computable por algún modelo de computación, entonces es computable en todos los otros.

Generalmente se considera una versión extendida de la tesis de Church-Turing:

*Si un problema es decidible en tiempo $O(t(n))$ por algún modelo razonable de computación (**determinista**), entonces es decidible en $O(t(n)^k)$ en cualquier otro modelo razonable de computación (**determinista**).*

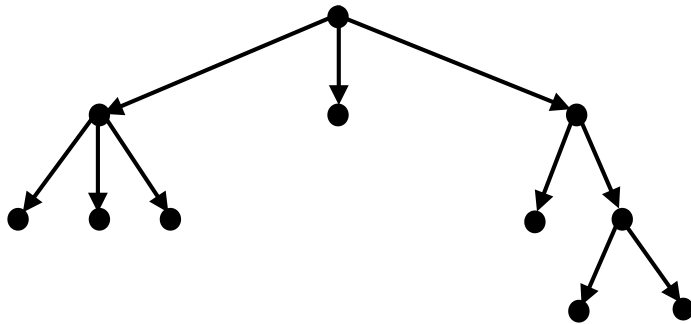
Tiempo de Ejecución en MT no-deterministas

Definición de tiempo de ejecución en máquinas de Turing no-deterministas



Simulando Máquinas No-Deterministas

Cualquier MT no-determinista que hace $O(t(n))$ pasos puede ser simulada por una MT determinista que hace $2^{O(t(n))}$ pasos.



Recordemos que la simulamos con un BFS, por niveles

- Como la MT termina, el árbol tiene a lo sumo $|b|^{t(n)}$ nodos
- En el peor caso los tenemos que recorrer todos,
- $b^{t(n)} \leq (2^c)^{t(n)} = 2^{c \cdot t(n)} \in 2^{O(t(n))}$

Clase P : Un Ejemplo

Veamos un ejemplo de problema en P :

Path = $\{\langle G, v, w \rangle \mid G \text{ es un grafo con un camino de } v \text{ a } w\}$

Algoritmo ineficiente:

Es $O(n!)$, siendo n la cantidad de nodos del grafo

Dado un grafo codificado con matriz de adyacencia:

Para $i=1,2,3,\dots,\#V$

- ➡ Se generan todos los posibles caminos de longitud i , con empiezan con v y terminan con w
- ➡ Si alguno de estos está en el grafo, se acepta

Si no se encontró, se rechaza.

Ejemplo (cont.)

Veamos una solución mejor:

Dado un grafo representado con matrices de adyacencia:

- ➡ Se marca v ,
- ➡ Mientras haya nodos sin marcar
 - ➡ Recorrer los arcos de G , para cada arco (a,b) con a marcado y b no, se marca b .
 - ➡ Si se marca w , se acepta.
- ➡ Se rechaza.

Se recorren todos los nodos, por cada uno se hace a lo sumo la cantidad de arcos, es decir, el algoritmo es:

$O(|V|*|E|)$ pasos, es decir, es **polinomial**.

Otro ejemplo

Veamos otro ejemplo en P :

$$\text{Coprimos} = \{ \langle x, y \rangle \mid x \text{ es coprimo de } y \}$$

Podríamos intentar el siguiente algoritmo:

- ➡ Dados dos números x e y en binario:
 - ➡ Para todo $z \leq \min(x, y)$ hacer
 - ➡ Si $z \mid x$ y $z \mid y$, terminar y rechazar
 - ➡ Sino aceptar

Los números vienen representados en binario. Es decir, si tenemos números de n bits, hacemos 2^n pasos

Otro Ejemplo (cont.)

Veamos otra posible implementación:

- ➡ Dados dos números x e y en binario:
 - ➡ Repetir hasta que $y = 0$
 - ➡ $x = x \bmod y$
 - ➡ intercambiar x e y
 - ➡ Si $x = 1$ aceptar, sino rechazar

Veamos el tiempo de ejecución.

- En cada paso x es dividido al menos por 2,
- Es decir, si los números tienen n bits, toma n pasos,
- La división se puede calcular en tiempo polinomial $p(n)$
- Es decir, es $n * p(n)$ que es polinomial.

Gramáticas Libre de Contexto

Veamos el siguiente ejemplo:

$$GLC = \{ \langle G, w \rangle \mid w \text{ es generado por la gramática libre de contexto } G \}$$

La idea es usar programación dinámica y la forma normal de Chomsky, es decir, producciones del estilo:

$$A \rightarrow BC \qquad A \rightarrow a \qquad A \rightarrow \epsilon$$

La idea es, dado $w = w_0 w_1 \dots w_{n-1}$, construir una matriz:

$$M[i][j] \text{ no terminal que genera desde } i \text{ a } j$$

Algoritmo

El algoritmo es el siguiente:

$D =$ “On input $w = w_1 \cdots w_n$:

1. For $w = \varepsilon$, if $S \rightarrow \varepsilon$ is a rule, *accept*; else, *reject*. $\llbracket w = \varepsilon \text{ case} \rrbracket$
2. For $i = 1$ to n : $\llbracket \text{examine each substring of length 1} \rrbracket$ n pasos
3. For each variable A : v variables
4. Test whether $A \rightarrow b$ is a rule, where $b = w_i$.
5. If so, place A in $table(i, i)$. n pasos
6. For $l = 2$ to n : $\llbracket l \text{ is the length of the substring} \rrbracket$
7. For $i = 1$ to $n - l + 1$: $\llbracket i \text{ is the start position of the substring} \rrbracket$ n pasos
8. Let $j = i + l - 1$. $\llbracket j \text{ is the end position of the substring} \rrbracket$
9. For $k = i$ to $j - 1$: $\llbracket k \text{ is the split position} \rrbracket$
10. For each rule $A \rightarrow BC$:
11. If $table(i, k)$ contains B and $table(k + 1, j)$ contains C , put A in $table(i, j)$.
12. If S is in $table(1, n)$, *accept*; else, *reject*.”

Este algoritmo es $O(|v| + n^2)$